# LAB SHEET 4

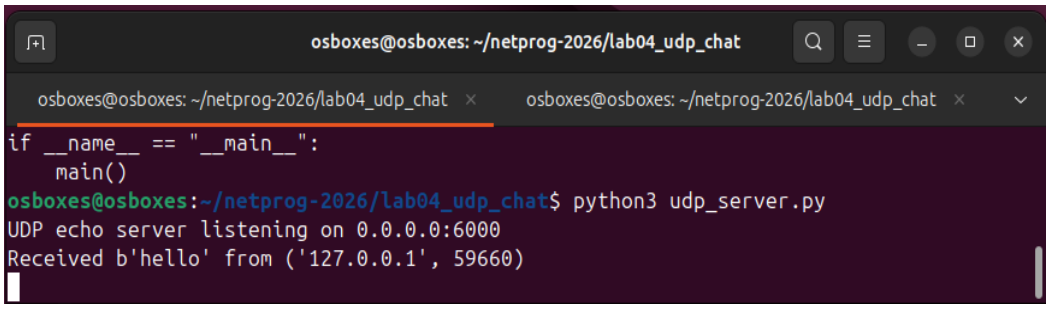
## Development Environment Setup & Core Networking Concepts in Practice

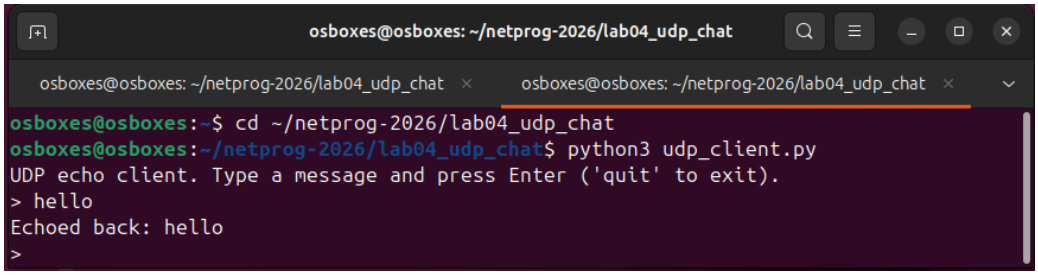
| | |
|---|---|
| **Week** | 3 |
| **Duration** | 2 hours |
| **Deliverable** | udp_server.py, udp_client.py, screenshot report (PDF), pushed to your netprog2026 GitHub repository or LMS2 |

### Lab Tasks

Complete the following tasks and record evidence (terminal output / screenshots) for each in your report.

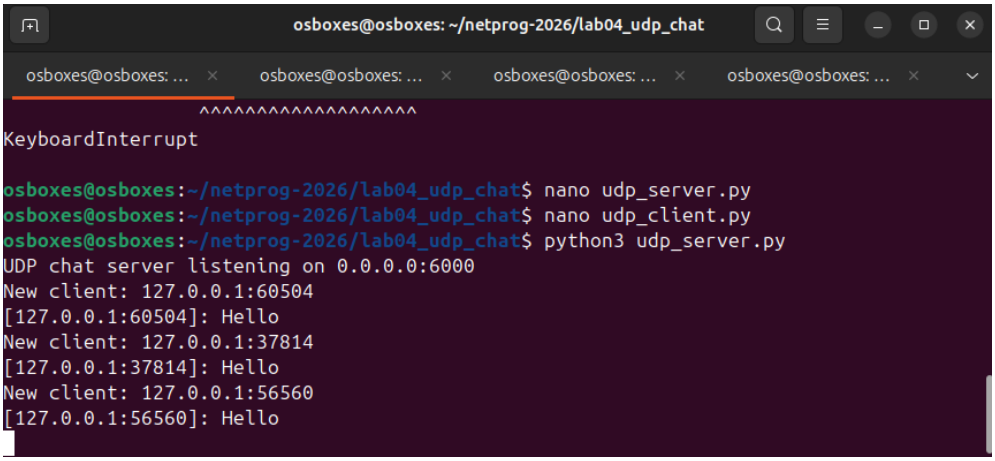
1. Task 1: Screenshot of udp_server.py and udp_client.py exchanging a single echoed message (Part D).



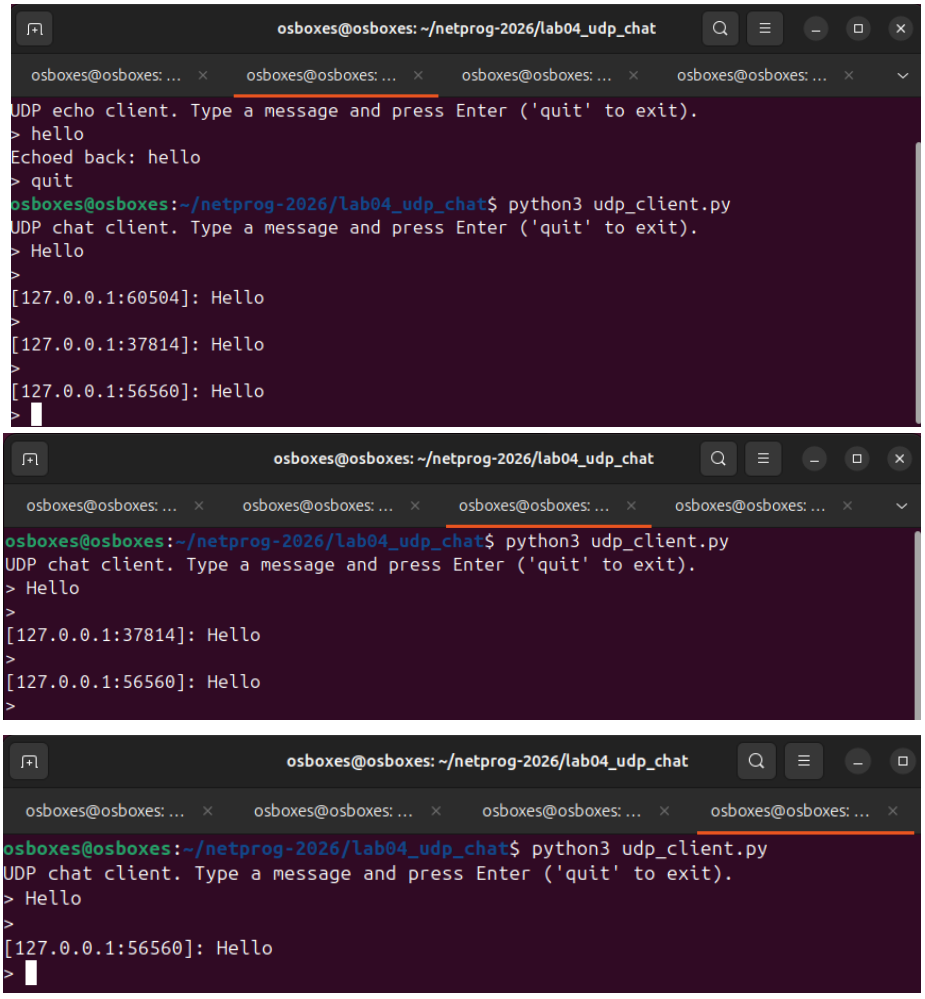



2. Task 2: Screenshot showing three chat clients (Part E) where a message sent from one client is broadcast to and displayed by all three.

UDP server

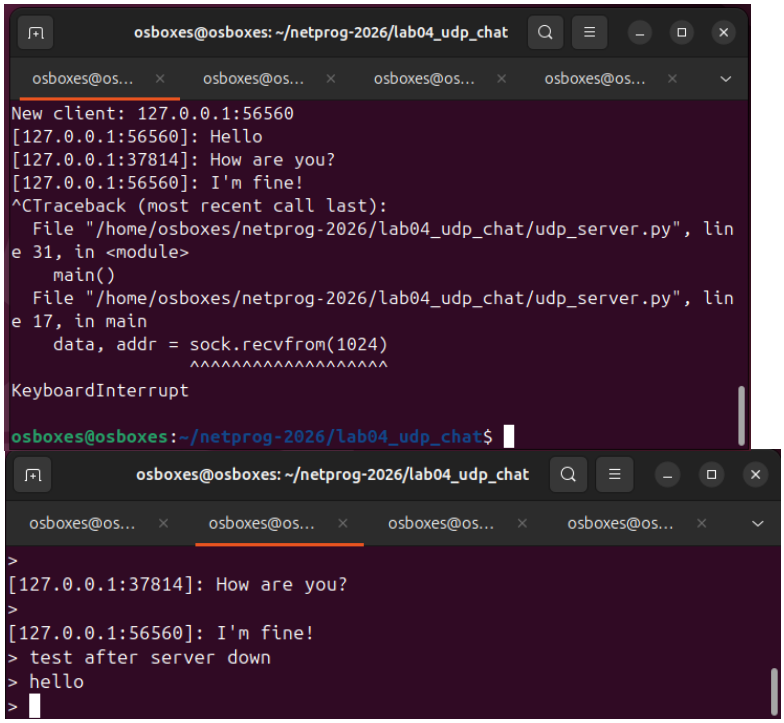



UDP Client

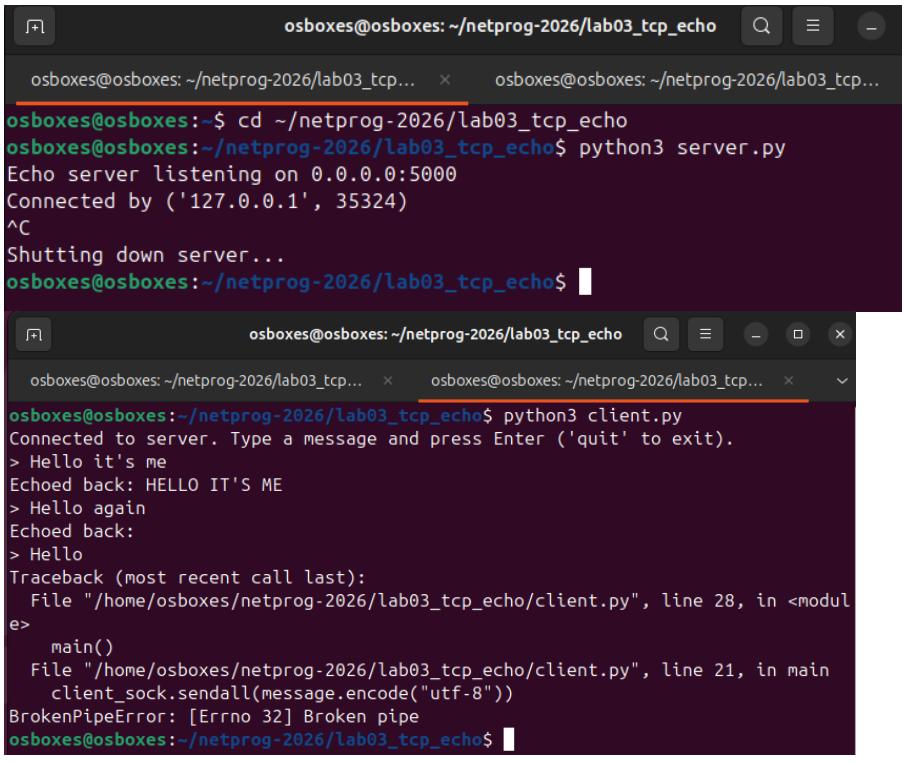


3. Task 3: Screenshot of Part F's server-down test for BOTH udp_client.py and Lab 3's client.py. In 2-3 sentences, explain the difference you observed and why it happens.

UDP Socket



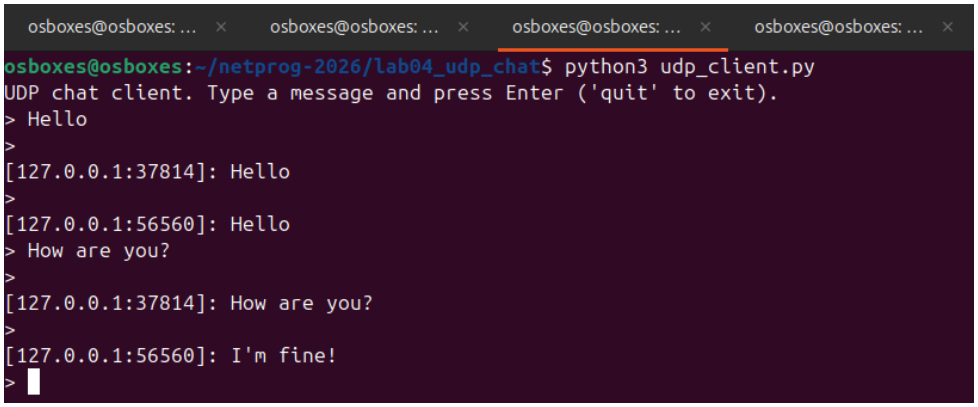

TCP Socket

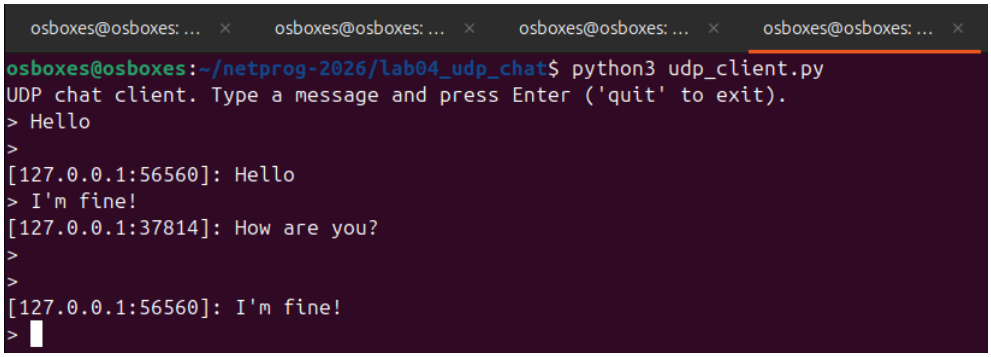


**Answer** TCP has a connection, so when the server process terminates, the OS closes the socket and sends a FIN or RST back to the client. The client can detect this on the next recv() or send() call. However, UDP has no connection. The sendto() function only sends a datagram to the kernel and then finishes. There is no acknowledgment. On Linux, an ICMP "port unreachable" message is not reported to a UDP socket that has not called connect(), so the client does not know that the message has been lost.

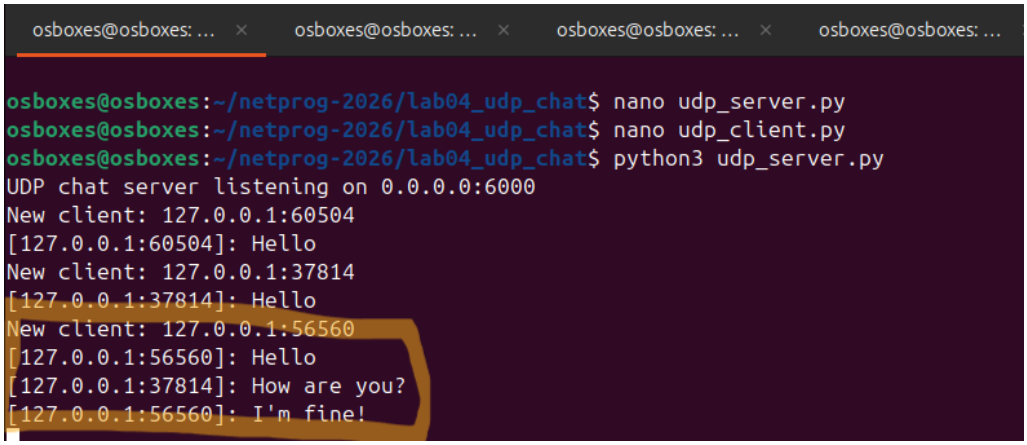
4. Task 4: Send messages from two clients at nearly the same instant (e.g., both presses Enter within a second of each other) and screenshot the order in which the server logs / broadcasts them. In 1-2 sentences, note whether the arrival order matched what you expected, and why UDP offers no guarantee either way.

2 clients at nearly the same instant:




The server logs / broadcasts:




**Answer** On localhost, the messages usually arrive in the same order as you press Enter because there is no real network in between. However, UDP has no sequence numbers and does not reorder data. On a real network, datagrams may take different routes, get delayed in queues, or be lost along the way. Therefore, there is no guarantee of message order.

5. Task 5: Modify the chat server so each broadcast message is prefixed with the sender's address, e.g., [127.0.0.1:51234]: hello (the reference format used throughout this lab). Screenshot the result.

```
osboxes@osboxes:~/netprog-2026/lab04_udp_chat$ python3 udp_server.py
UDP chat server listening on 0.0.0.0:6000
New client: 127.0.0.1:60504
[127.0.0.1:60504]: Hello
New client: 127.0.0.1:37814
[127.0.0.1:37814]: Hello
New client: 127.0.0.1:56560
[127.0.0.1:56560]: Hello
```

6. Task 6: In 2-3 sentences, describe one real application where UDP's lack of delivery/ordering guarantees is an acceptable trade-off (or even beneficial), and explain why.

**Answer** Online multiplayer games often use UDP to send player positions many times per second. If one update is lost or arrives out of order, the game simply uses the next, newer update, so the old one is no longer needed. This keeps the game fast and responsive, while TCP's waiting and retransmission would cause noticeable lag.

## Troubleshooting Tips

- python3: command not found (Windows) — use py or python instead, or reinstall Python with "Add to PATH" checked.
- socket.gaierror: [Errno -2] Name or service not known — check your internet connection and that you typed the hostname correctly (no https:// prefix).
- netstat / ss shows nothing — make sure connect_demo.py is still running (it sleeps for only 5 seconds) when you check in the second terminal.
- http_probe.py hangs instead of returning — you likely forgot Connection: close in the request, so the server keeps the connection open waiting for another request.
- ConnectionRefusedError immediately in Part F step 2 instead of a hang — some networks/routers reject unreachable private addresses instantly rather than dropping packets silently; note this in your report and try a different unreachable address if you want to see the full hang.

## Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1. Explain how recvfrom() differs from recv(), and why UDP's API has no listen() or accept() at all.

   **Answer** recv() is used on a TCP socket that is already connected to one specific partner, so it only returns the data. recvfrom() returns both the data and the address of the sender, because on a UDP socket each datagram can come from anyone. UDP has no listen() or accept() because it never sets up a connection. There is no handshake to wait for, so one socket can receive datagrams from every client directly.

2. Why can a UDP server “broadcast” to multiple clients just by remembering their addresses, while a TCP server needs a separate persistent connection (and usually a separate thread) per client to do the same thing?

   **Answer** In UDP, every datagram carries the sender’s address, and sendto() can send to any address at any time. The server only needs to keep a list of addresses and loop through it with one socket. In TCP, each client gets its own socket from accept(), and data can only go through that open connection. Because recv() blocks while waiting on one client, the server usually needs a separate thread per client to handle everyone at the same time.

3. Give an example of a real application where occasional packet loss is acceptable and explain why UDP's lack of retransmission is actually a reasonable trade-off there rather than a flaw.

   **Answer** Video calls such as Zoom or Google Meet are a good example. If a few packets are lost, you might see a small glitch in the picture or hear a short gap in the sound, and the call just continues. If the app waited to resend lost packets, the late data would arrive too late to be useful and would make the whole call lag. Skipping lost packets keeps the conversation in real time, which matters more than perfect quality.